pfi

Python for Fixed Income

COURSE 5 · ADVANCED MACHINE LEARNING

Bagging and Out-of-Bag Scores

WEEK 1 · DAY 2

Created by **Jeff Lenamon**

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

Bagging, and a score that costs no rows

1. Draw

A bootstrap sample of the training rows

2. Fit

One full tree on it

3. Average

Repeat, and average the trees' outputs

Today's two questions

How many trees before more stop helping?

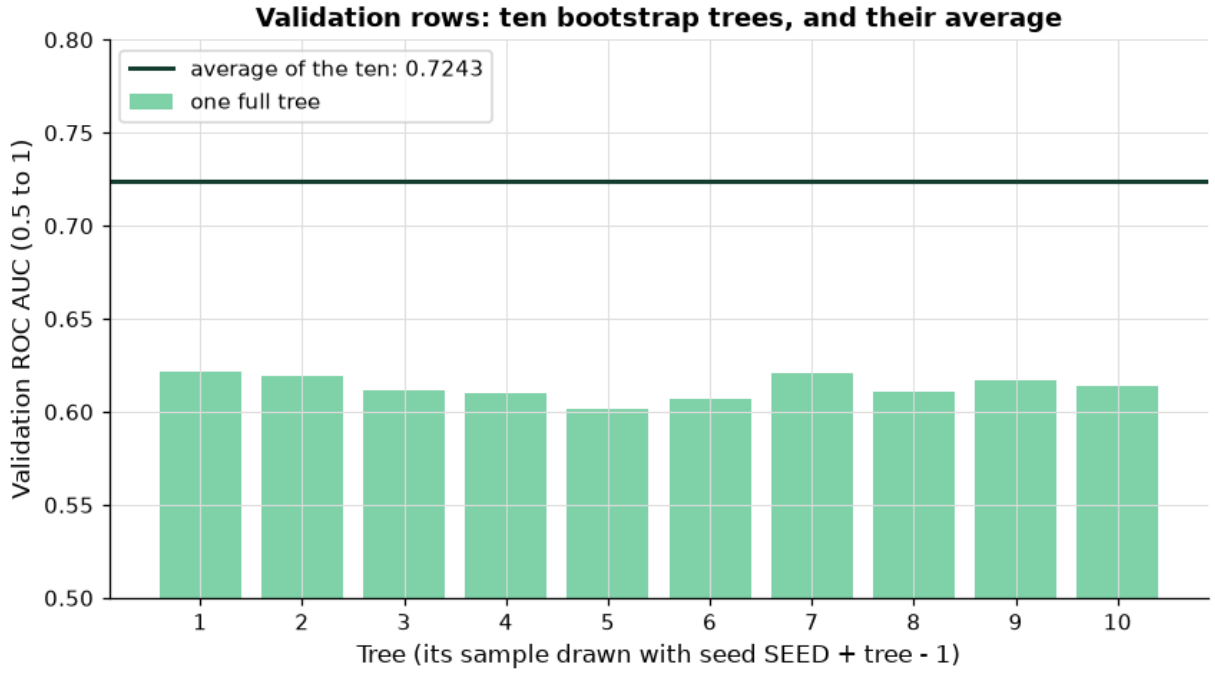
Can the rows each tree never saw score the whole model, and what does scikit-learn's ready-made score measure?

Bagging helps unstable models. A full tree is as unstable as models come.

How bagging works

The problem	A full tree is unstable: change the training rows a little and the tree changes
Mechanism	One bootstrap sample and one full tree per ensemble member; average the class-1 probabilities
Why it helps	Each tree has seen different rows, so each errs in different places; the average keeps what they share
You control	The number of trees, the model repeated, and the seed
What comes out	One model probability of default in this file per account

A bagged model is the average of its trees



0.6015 to 0.6219

single full trees, validation ROC AUC

0.7243

their average, by hand

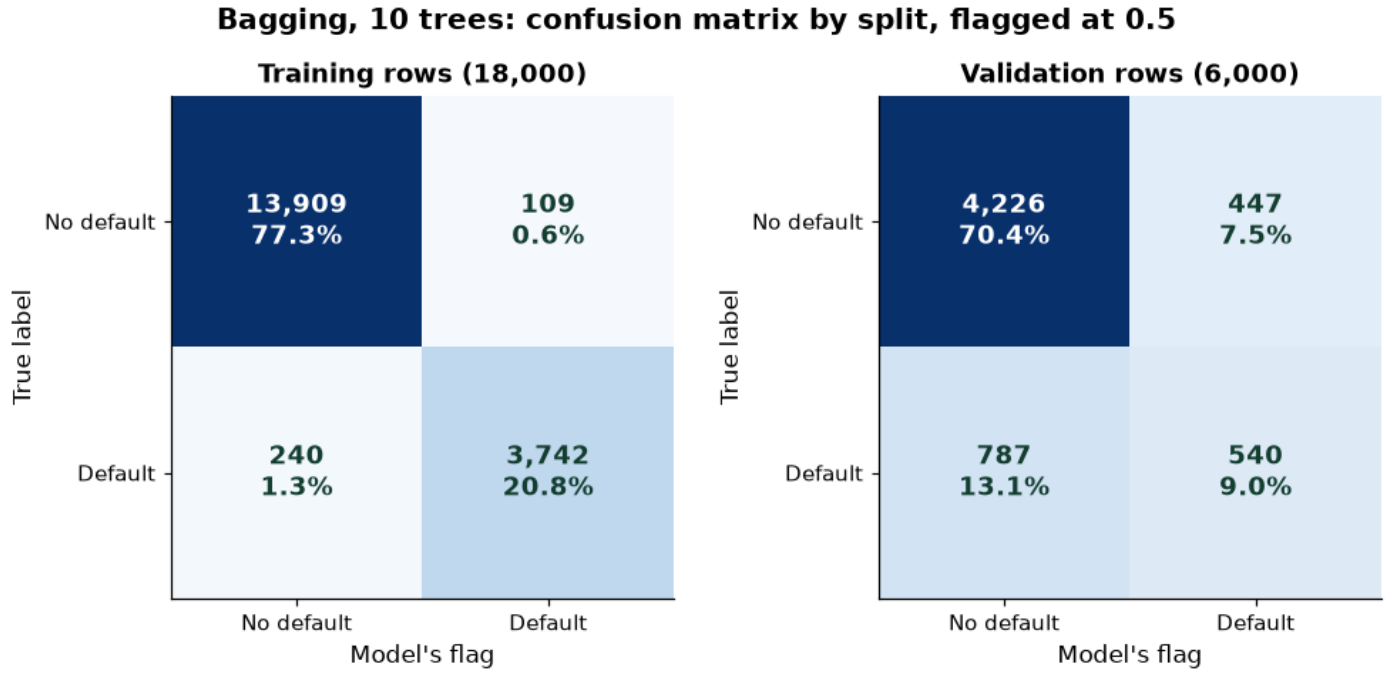
0.7320

BaggingClassifier, 10 trees (its own samples)

```
np.abs(ensemblekit.bagged_proba(bagging_10, X_valid) - proba_10).max() < 1e-12   # True
```

Each tree sees 18,000 draws holding 11,321 to 11,423 distinct rows. The output is their plain average.

Training rows flatter the trees



0.9397

recall, training rows

0.4069

recall, validation rows

0.9806 / 0.7943

accuracy, training / validation

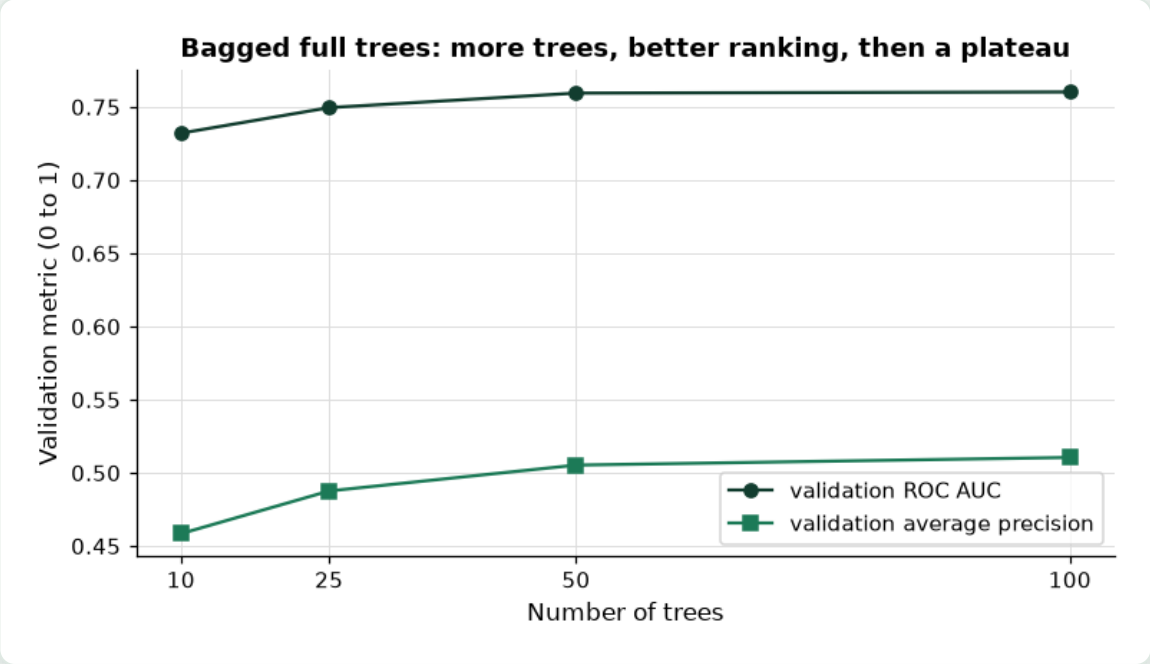
A full tree returns its own rows almost exactly, so the training rows say nothing about new accounts.

How BaggingClassifier works

The problem	Writing the bootstrap loop by hand for every model is slow and easy to get wrong
Per tree	Draw a bootstrap sample, fit a copy of your model on it, keep the copy in estimators_
Predict	predict_proba asks every tree for its class-1 probability and averages
You control	The model to repeat, n_estimators, and random_state
What comes out	A fitted object whose predict_proba is that average; the next function checks it

More trees, better ranking, then a plateau

Trees	Validation ROC AUC	Average precision
10	0.7320	0.4584
25	0.7495	0.4876
50	0.7594	0.5053
100	0.7602	0.5105



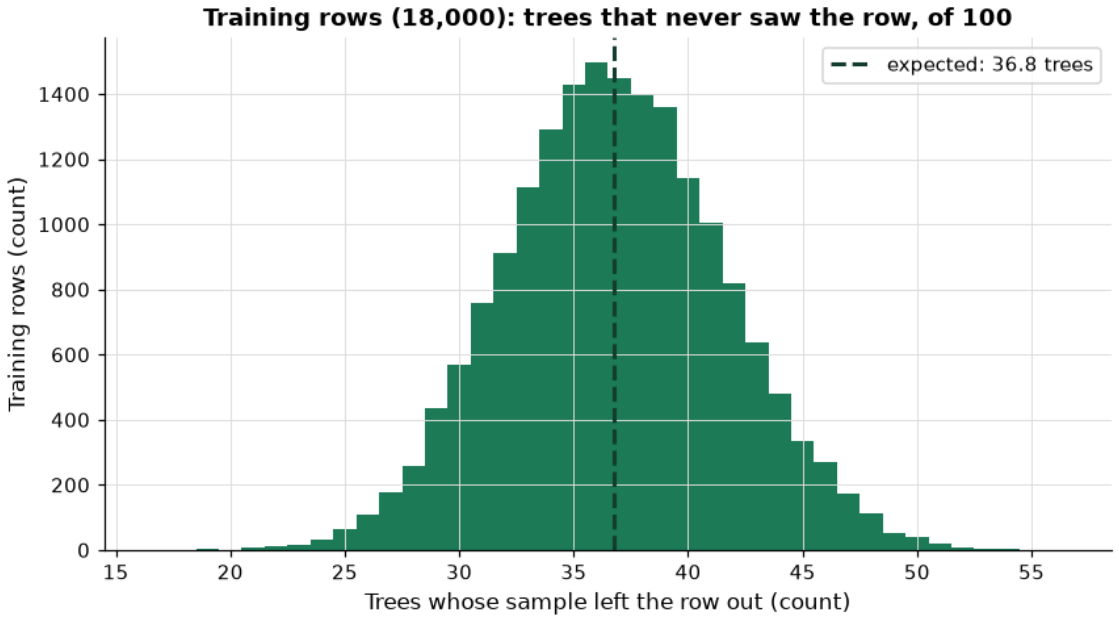
10 to 25 trees: +0.0175. 50 to 100 trees: +0.0008. Logistic regression: 0.7190.

How the out-of-bag score works

The problem	Every training row was used to fit the trees, so the trees flatter themselves on those rows
Left out	A row sits outside about 36.8 percent of the bootstrap samples, so a third of the trees never saw it
Average	For each training row, average only the trees that never saw it
You control	oob_score=True and the number of trees
What comes out	One honest output per training row: a validation set that costs no rows

Every training row, scored by trees that never saw it

Each tree's sample leaves a row out with chance about 36.8 percent. Average only those trees' outputs for that row: an honest output, from one fit.



36.8

trees that never saw a row, on average

17 to 56

fewest to most: no row at zero

True

same trees as without oob_score=True

11 tests pass at the end of today.

oob_score_ is accuracy, not ROC AUC

0.8109

oob_score_ (accuracy)

0.7788

flag no account

0.7502

out-of-bag ROC AUC

0.7602

validation ROC AUC

A high floor

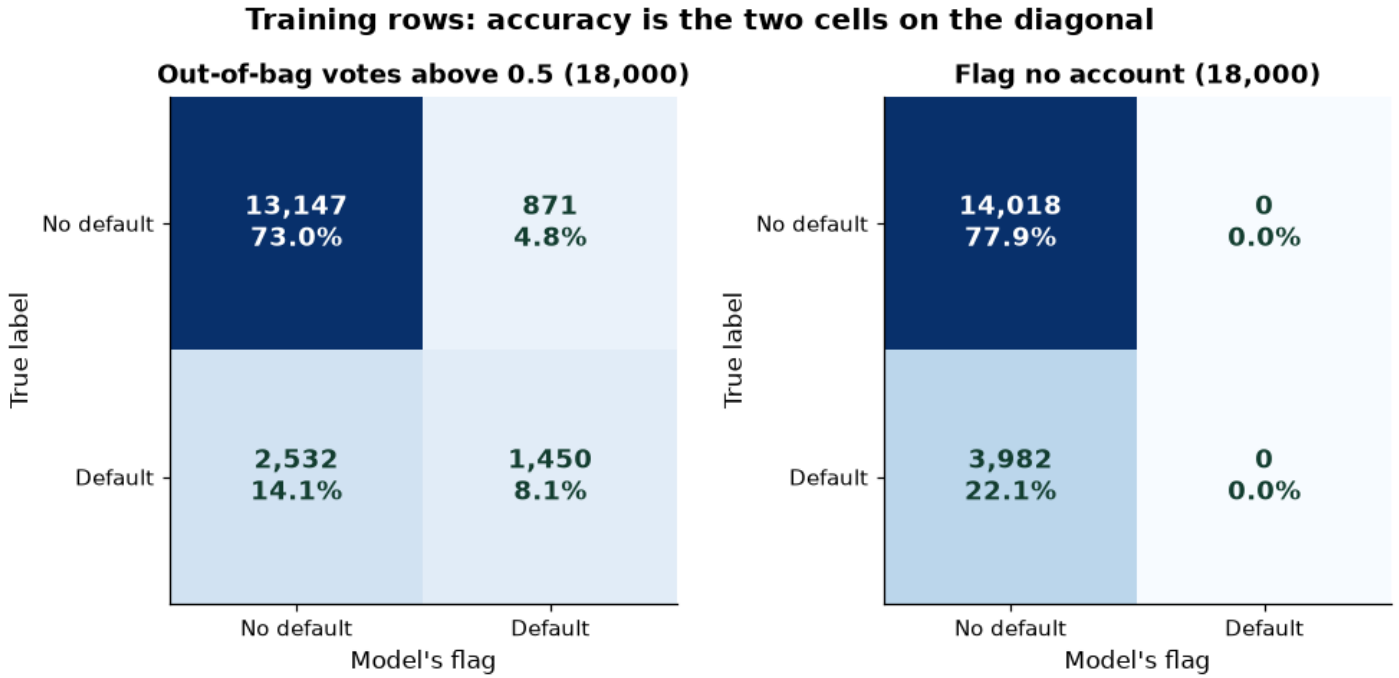
Most accounts do not default, so saying "no default" is right 0.7788 of the time. oob_score_ is only a little above it.

Why it leans low

Each row's out-of-bag output averages about 37 trees, not 100. The 25-tree model scored 0.7495 on validation.

Compare models by ensemblekit.oob_auc, never by oob_score_ beside a ROC AUC.

Accuracy is the diagonal: a high floor

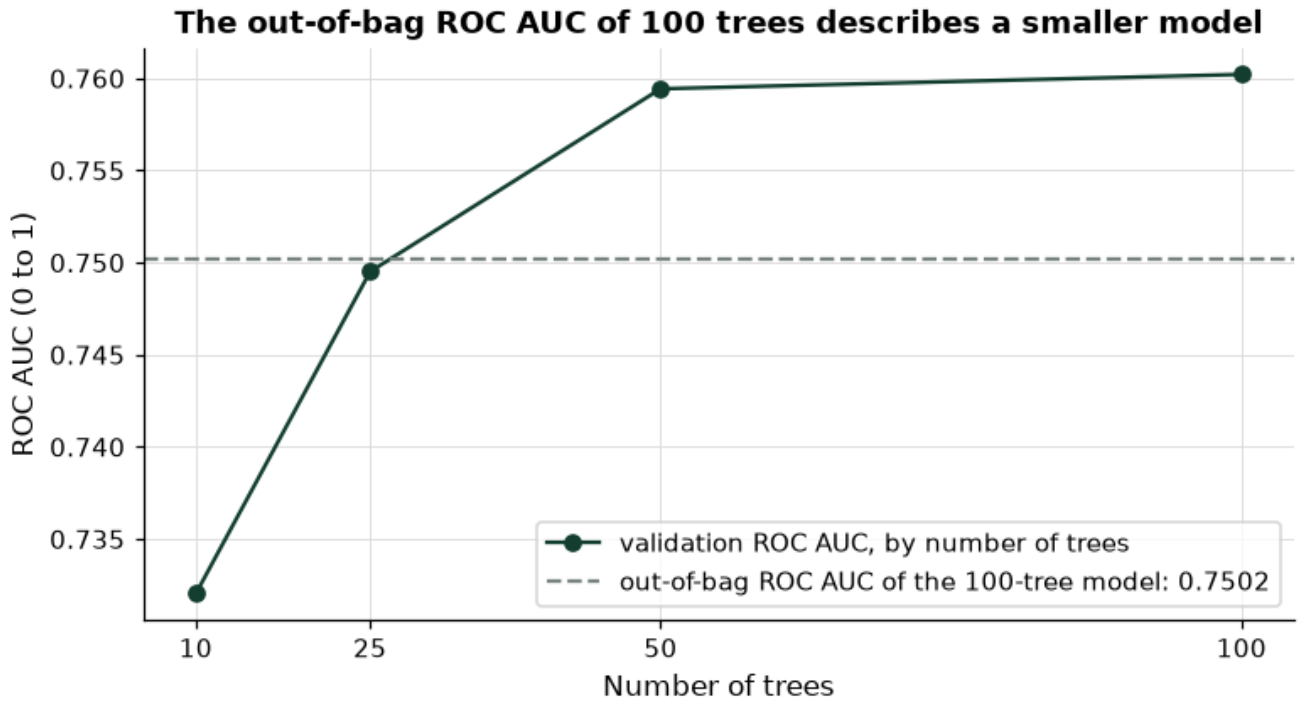


Read the two diagonals

Flag no account: 0.7788, without a single default found. Out-of-bag votes: 0.8109. Most of that accuracy is the top-left cell, which flagging nobody gets for free.

oob_score_ belongs beside other accuracies, never beside a ROC AUC.

The out-of-bag ROC AUC describes a smaller model



0.7502

out-of-bag ROC AUC, 100 trees

0.7495

validation, 25 trees

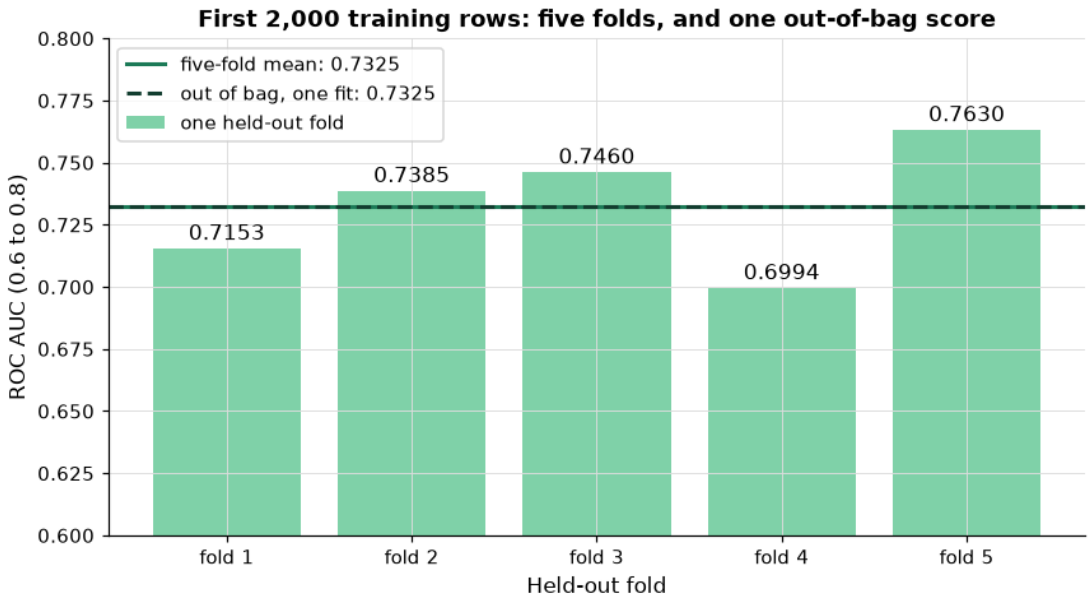
0.7594

validation, 50 trees

Each row's out-of-bag output averages about 37 trees, and it scores like a model of about that size.

Where out-of-bag helps: a small file

The first 2,000 training rows only, 415 defaults. A 100-tree bagged model, scored two ways.



0.7325

out of bag, one fit

0.7325

five-fold mean, five fits

0.6994 to 0.7630

the five folds (sd 0.0225)

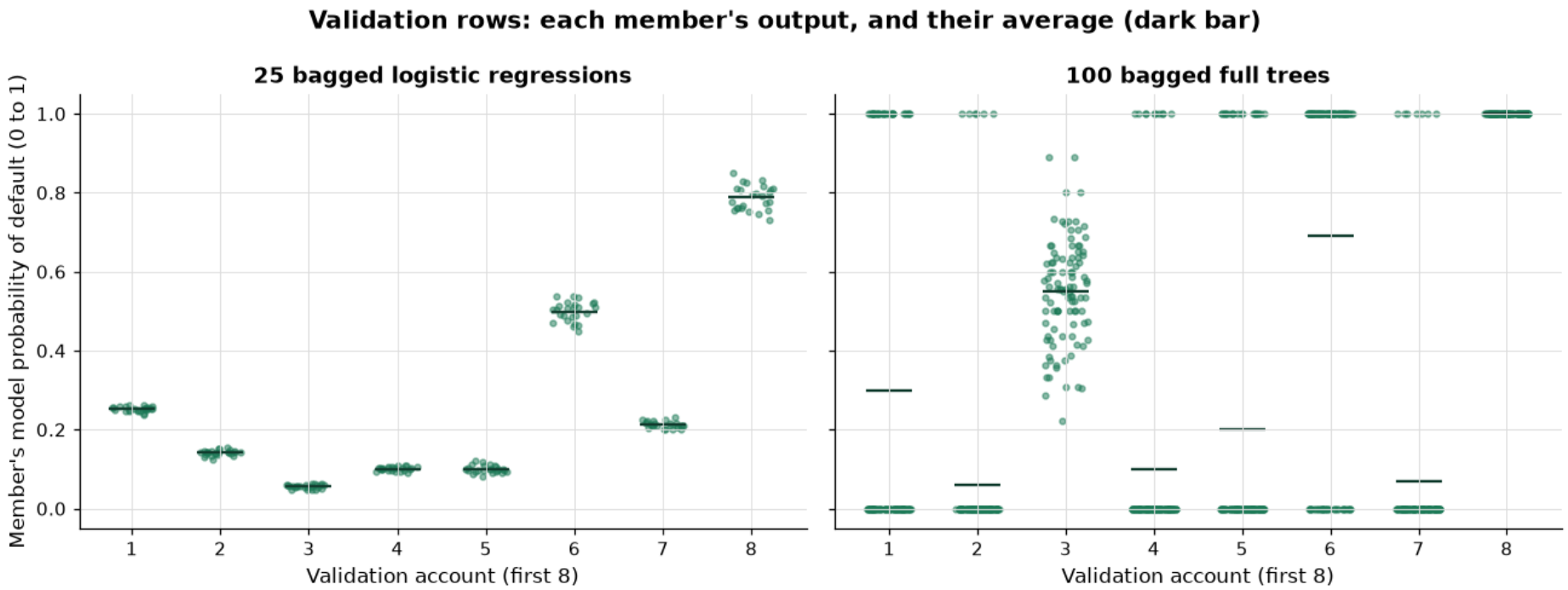
On a bagged model or a forest, the out-of-bag ROC AUC is a fair stand-in for cross-validation when rows are few. Validation rows: 0.7366.

Bagging a stable model changes almost nothing

	Logistic regression	Full trees
Validation ROC AUC, one model	0.7190	0.6015 to 0.6219
Validation ROC AUC, bagged	0.7195 (25 copies)	0.7602 (100 trees)
Average spread of members on an account	0.0121	0.3365

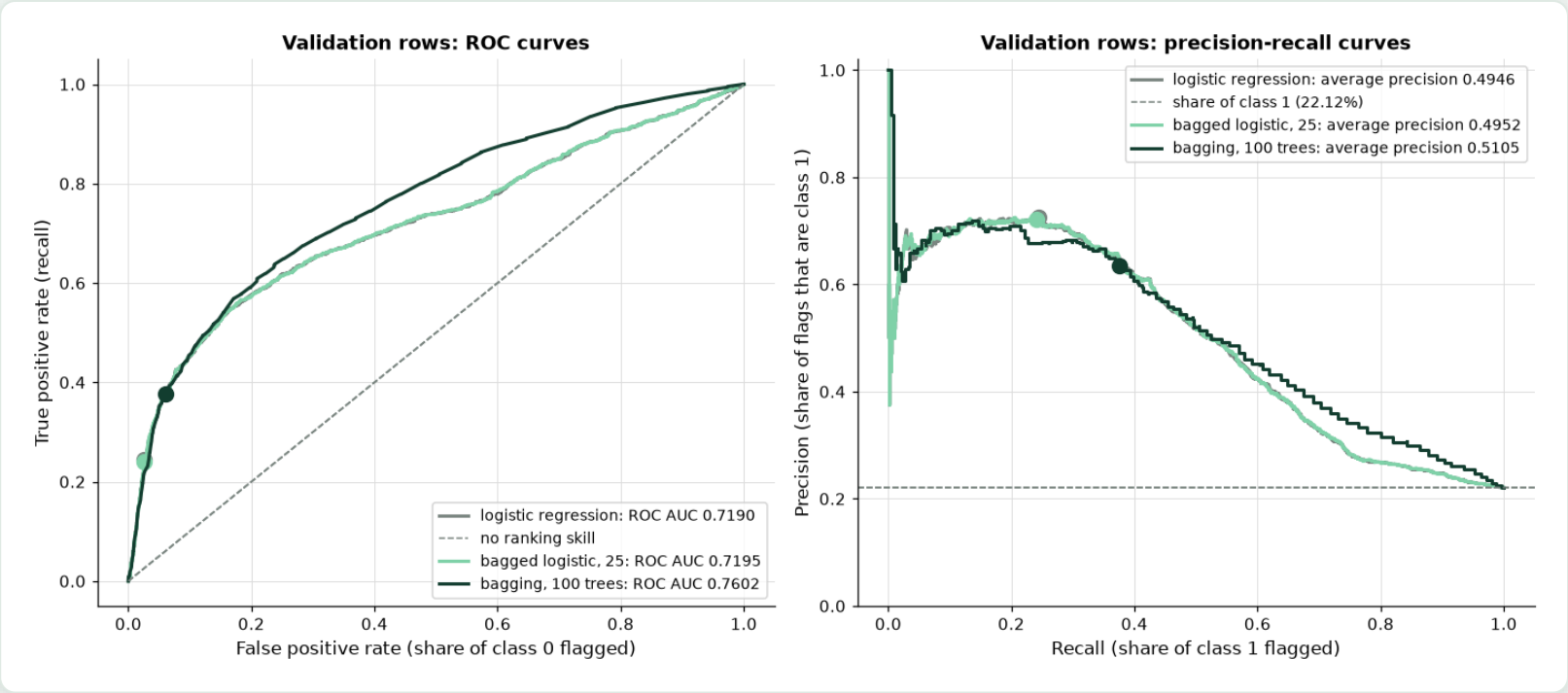
Members that barely disagree leave nothing to average away. Bagging helps unstable models.

The members, account by account



Left: 25 logistic regressions, one tight cluster per account (average spread 0.0121). Right: 100 trees from 0 to 1 (0.3365); only their average settles.

Three models, one chart



Validation rows	ROC AUC	AP
Logistic	0.7190	0.4946
Bagged logistic	0.7195	0.4952
Bagged trees	0.7602	0.5105

The two logistic curves overlap. The dots mark each model flagged at 0.5; the trees flag more defaults there.

Bagging lifts the unstable model and leaves the stable one where it was. AP: average precision.

What bagging cannot fix

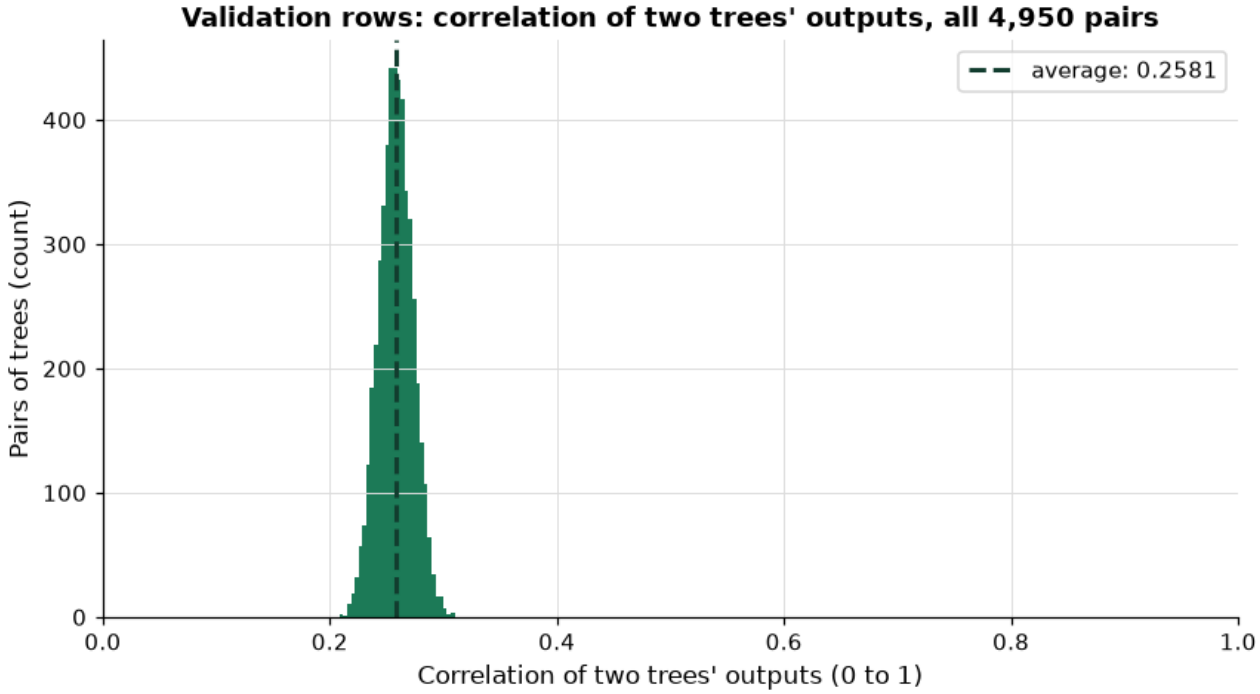
First question of the 100 trees

```
PAY_0 <= 1.5: 99 trees

PAY_0 <= 0.5: 1 tree
```

0.2581

average correlation of two trees' outputs (4,950 pairs)



The part every tree shares cannot average away. Day 3: let each split see only a few columns, so the trees disagree.

Every setting of BaggingClassifier

Setting	Default	Tune first?
estimator	None	tune first
n_estimators	10	
max_samples	None	tune first
bootstrap	True	
max_features	1.0	tune first
bootstrap_features	False	
oob_score	False	
warm_start	False	
n_jobs	None	
random_state	None	
verbose	0	

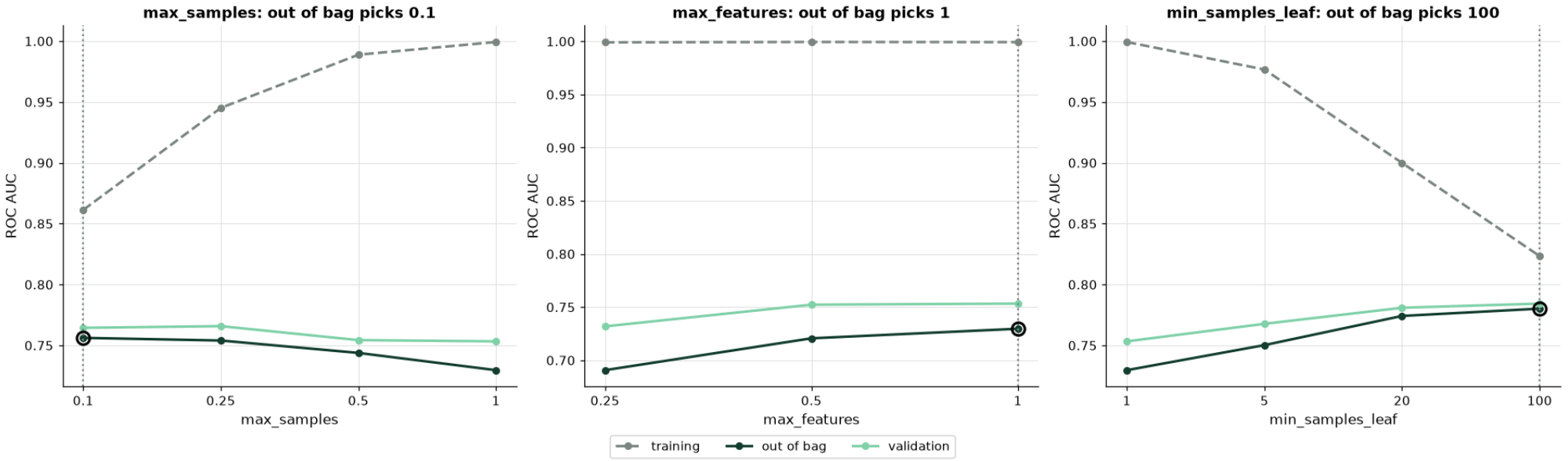
Read them by group

What is repeated (estimator). How many (n_estimators). What each model sees: rows (max_samples, bootstrap) and columns (max_features, bootstrap_features). Scoring, speed, and the seed.

The three to tune first

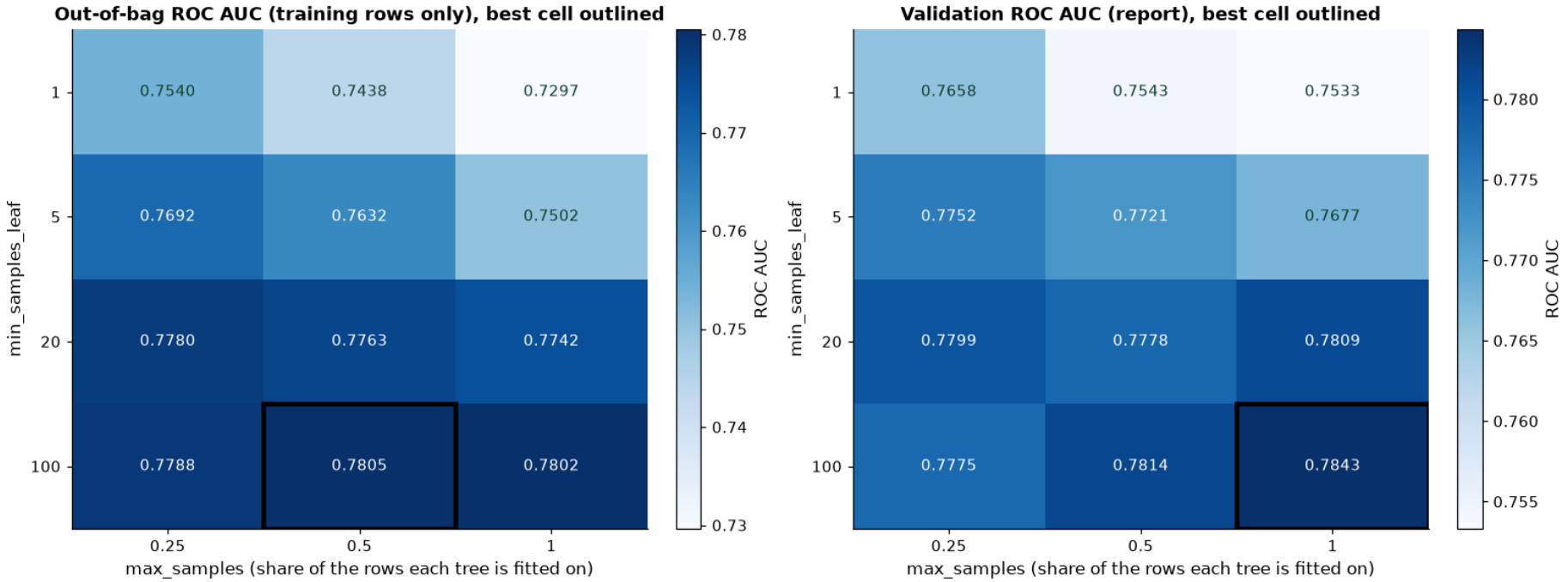
max_samples, max_features, and the settings of the tree inside. They decide how strong the members are and how alike. Built from the installed library's signature, so the defaults are never typed.

One setting at a time, chosen out of bag



Setting	Chosen	Training	Out of bag	Validation
max_samples	0.1	0.8613	0.7562	0.7644
max_features	1	0.9993	0.7297	0.7533
min_samples_leaf	100	0.8234	0.7802	0.7843

Two settings together: sample size and leaf size



Both settings fight trees that remember their rows: once one has done the work, the other has little left to do.

Excel side by side

Excel side by side (tested in Excel)

`=AVERAGE(B2:K2), filled down`	equals predict_proba on all 6,000 accounts
`=SUMPRODUCT(--((B2:B18001>0.5)=(A2:A18001=1)))/ROWS(A2:A18001)`	0.8109: oob_score_ exactly
`>=0.5 in place of >0.5`	0.8099: ties at 0.5 count as no default
`=1-AVERAGE(A2:A18001)`	0.7788, the all-zero baseline
`=SORT(UNIQUE(B2:B18001),,-1), rates by MAP(LAMBDA), trapezoid`	0.7502, within 1e-6

No bagging and no tree in Excel. 10 of 11 near-equal out-of-bag pairs compared as equal in Excel, hence 1e-6.

Save the day: git diff on the log

1. Code

mlkit.py, ensemblekit and its tests

2. Log

results.csv with the 10-tree model

3. Diff

two rows added: + marks them

4. Commit

the log, three commits in all

```
bagging_10_trees,validation,0.7320,0.4584,0.5000,0.7943,0.5471,0.4069,0.4667
+bagging_100_trees,validation,0.7602,0.5105,0.5000,0.8143,0.6357,0.3760,0.4725
+bagged_logistic_25,validation,0.7195,0.4952,0.5000,0.8117,0.7223,0.2411,0.3616
```

Four decimals and LF endings: the unchanged row shows no mark, so a diff shows only real changes.

The AI check

The docstring is the prompt. The code that comes back runs, and its number looks fine.

1

Docstring

Which rows fit, which rows choose, which rows report: what you asked for.

→

2

Code back

Written for the course in the style of an AI answer. Read it.

→

3

Test first

Assert what the docstring promises. Run it: it fails.

→

4

Fix

Find the rule it broke, fix it, run the test again.

Today: oob_score_, an accuracy, is returned as the out-of-bag ROC AUC.

Code that runs is not the same as code that is right.

What you can do now

- BaggingClassifier is the plain average of its trees
- More trees help, then the curve flattens
- oob_score_ is accuracy; oob_auc is the ranking number
- Out of bag stands in for cross-validation on few rows
- Bagging helps unstable models, not a logistic regression

Week 1, Day 3 Random forests: trees that disagree on purpose